FPT SCHOOL OF BUSINESS & TECHNOLOGY

AI - MACHINE LEARNING

PhD. Truc Thi Kim, Nguyen

093 293 9981 info@fsb.edu.vn https://fsb.edu.vn

Shallow Neural Network

FSB

Shallow Neural Network

- If we have m training examples we need to repeat this process m times. For each training example, or for each feature vector that looks like this:

$$z^{[1]} = \mathbf{W}^{[1]}x + b^{[1]}$$

$$a^{[1]} = \sigma(z^{[1]})$$

$$z^{[2]} = \mathbf{W}^{[2]}a^{[1]} + b^{[2]}$$

$$a^{[2]} = \sigma(z^{[2]})$$

$$x \rightarrow a^{[2]} = \hat{y}$$

$$x^{(1)} \rightarrow a^{[2](1)} = \hat{y}$$

$$x^{(2)} \rightarrow a^{[2](2)} = \hat{y}$$

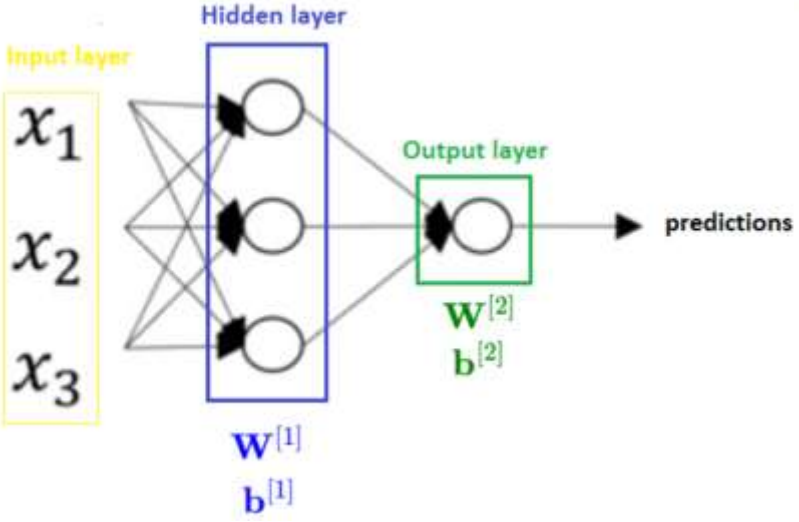
$$\vdots$$

$$x^{(m)} \rightarrow a^{[2](m)} = \hat{y}$$

FSB

Shallow Neural Network

- m training examples:



$$for\ i\ in\ range(m):$$

$$z^{[1](i)} = \mathbf{W}^{[1]}x^{(i)} + b^{[1]}$$

$$a^{[1](i)} = \sigma(z^{[1](i)})$$

$$z^{[2](i)} = \mathbf{W}^{[2]}a^{[1](i)} + b^{[1]}$$

$$a^{[2](i)} = \sigma(z^{[2](i)})$$

FSB

Shallow Neural Network Vectorization

- Matrix X is a feature vector for one training example, so the dimension of this matrix is (*number of features in every vector, number of training examples*).

$$\mathbf{X} = \begin{bmatrix} | & | & \cdots & | \\ x^{(1)} & x^{(2)} & \cdots & x^{(m)} \\ | & | & \cdots & | \end{bmatrix}$$

$$\mathbf{Z}^{[1]} = \begin{bmatrix} | & | & \cdots & | \\ z^{[1](1)} & z^{[1](2)} & \cdots & z^{[1](m)} \\ | & | & \cdots & | \end{bmatrix}$$

$$\mathbf{A}^{[1]} = \begin{bmatrix} | & | & \cdots & | \\ a^{[1](1)} & a^{[1](2)} & \cdots & a^{[1](m)} \\ | & | & \cdots & | \end{bmatrix}$$

$$\mathbf{Z}^{[1]} = \mathbf{W}^{[1]}\mathbf{X} + b^{[1]}$$

$$\mathbf{A}^{[1]} = \sigma(\mathbf{Z}^{[1]})$$

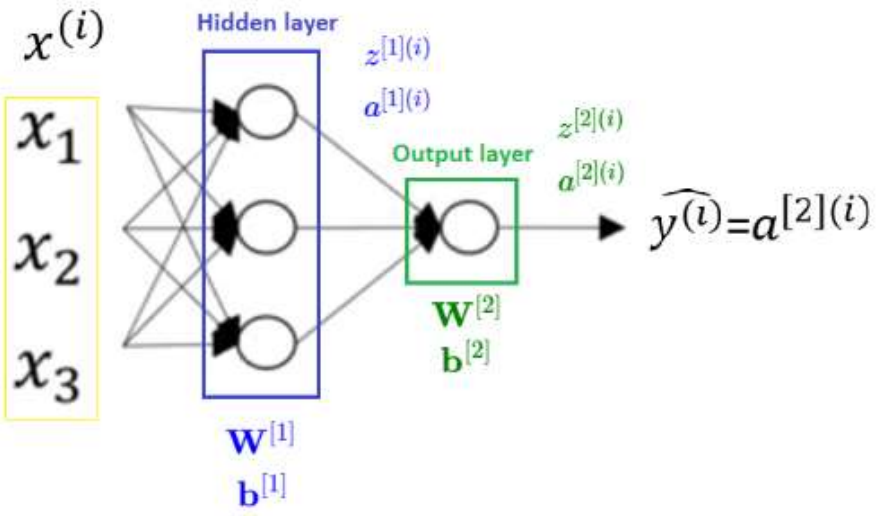
$$\mathbf{Z}^{[2]} = W^{[2]}\,\mathbf{A}^{[1]} + b^{[2]}$$

$$\mathbf{A}^{[2]} = \sigma(\mathbf{Z}^{[2]})$$

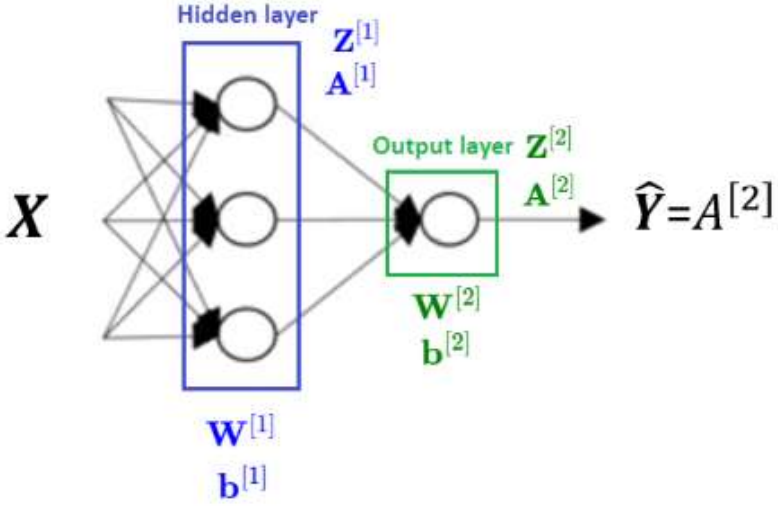
FSB

Shallow Neural Network Vectorization

Non - Vectorized

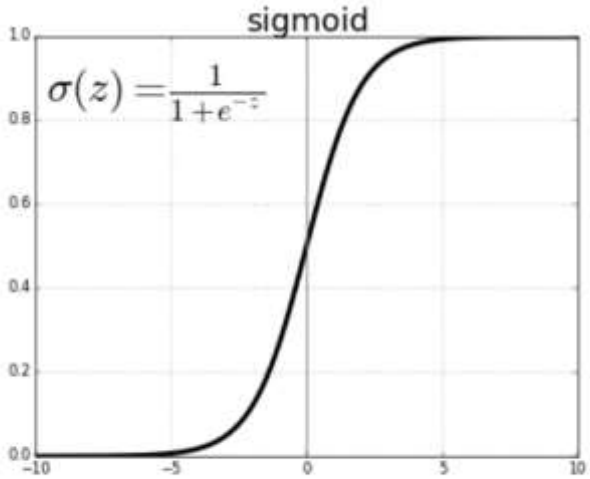


Vectorized

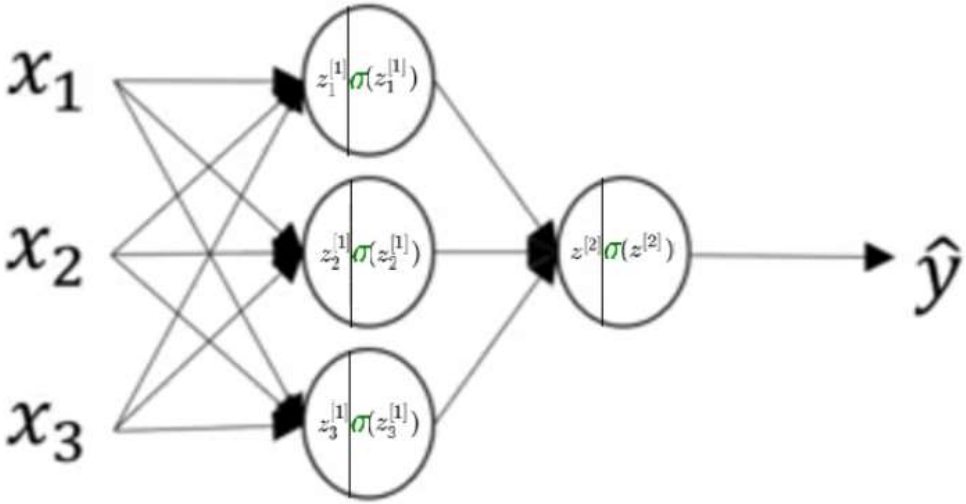


Activation functions and their derivatives

- Sigmoid activation function

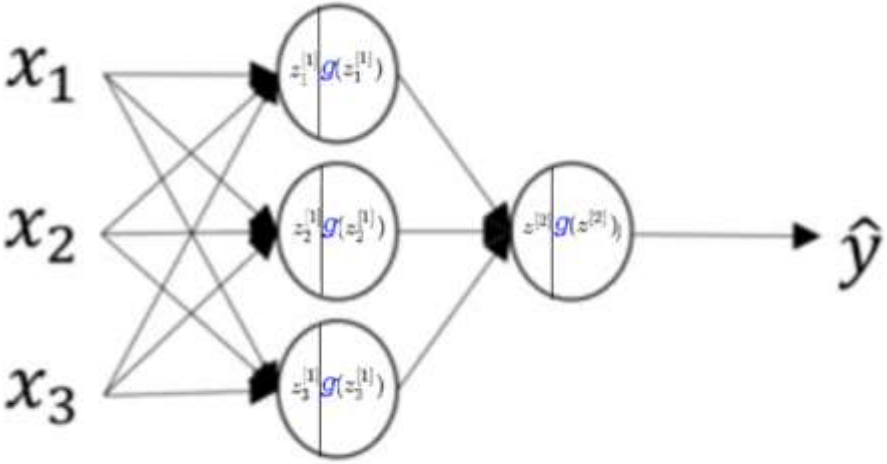


sigmoid function



A 2-layer Neural Network with sigmoid activation function in both layers

- General case we can have a different function which we will denote with g(z).



A 2-layer Neural Network with activation function \(g({z})\)

$$z^{[1]} = W^{[1]}x + b^{[1]}$$

$$a^{[1]} = g(z^{[1]})$$

$$z^{[2]} = W^{[2]}a^{[1]} + b^{[1]}$$

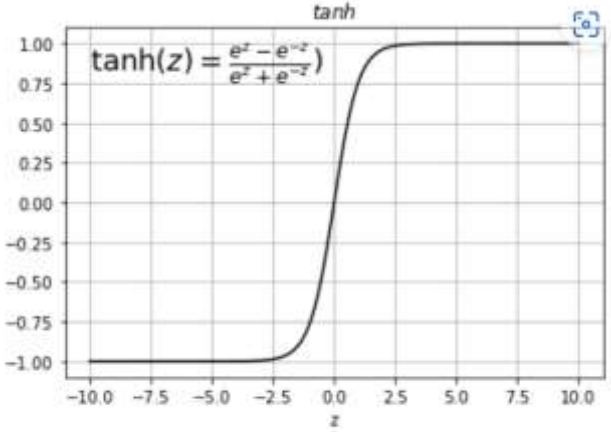
$$a^{[2]} = g(z^{[2]})$$

FSB

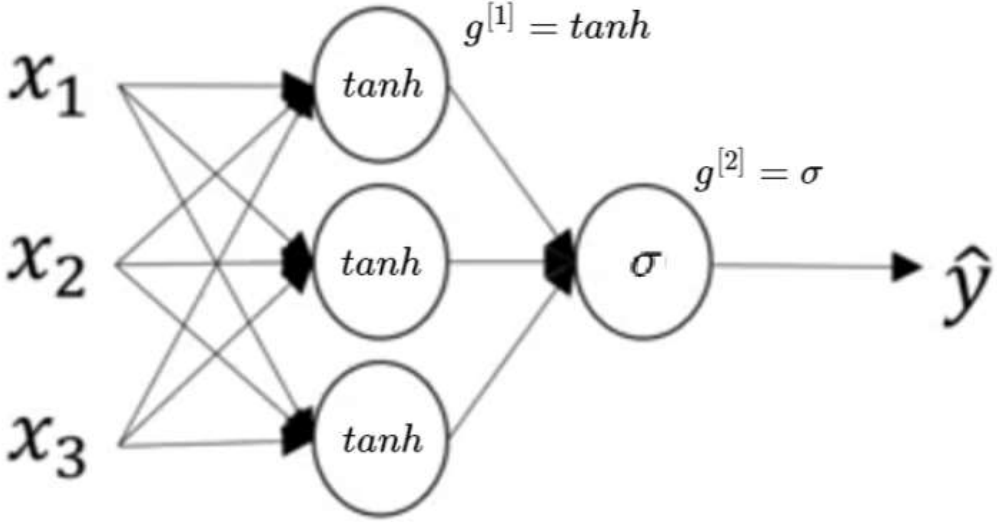
Activation functions and their derivatives

Tanh activation function

- A shifted version of a sigmoid function but scaled between −1 and 1.
- A 2-layer Neural Network with tanh activation function in the first layer and sigmoid activation function in the second layer.



tanh activation function



- Derivative of sigmoid function

$$g(z) = \frac{1}{1+e^{-z}}$$

$$\frac{d}{dz}g(z) = slope\ of\ g(z)\ at\ z$$

$$\frac{d}{dz}g(z) = \frac{-1}{(1+e^{-z})^2}e^{-z}(-1) = \frac{e^{-z}}{(1+e^{-z})^2} = \frac{e^{-z}+1-1}{(1+e^{-z})^2}$$

$$\frac{d}{dz}g(z) = \frac{1}{1+e^{-z}}\left(\frac{1+e^{-z}}{1+e^{-z}} + \frac{-1}{1+e^{-z}}\right) = \frac{1}{1+e^{-z}}\left(1 - \frac{1}{1+e^{-z}}\right) = g(z)(1-g(z))$$

$$g'(z) = g(z)(1-g(z))$$

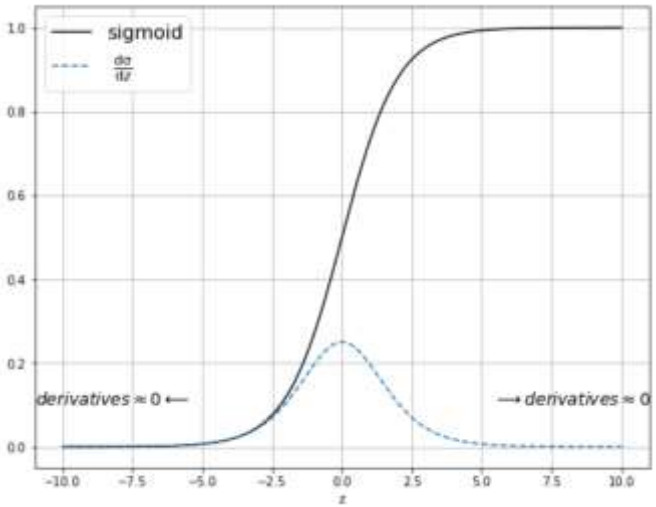
$$z = 10 \quad g(z) \approx 1 \Rightarrow g'(z) \approx 1(1-1) \approx 0$$

$$z = -10 \quad g(z) \approx 0 \Rightarrow g'(z) \approx 0(1-0) \approx 0$$

$$z = 0 \quad g(z) = \frac{1}{2} \Rightarrow g'(z) = \frac{1}{2} = \left(1 - \frac{1}{2}\right) = \frac{1}{4}$$

$$a = g(z) = \frac{1}{1+e^{-z}}$$

$$\frac{d}{dz}g(z) = a(1-a)$$



- Derivative of a tanh function

$$g(z) = tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

$$\frac{d}{dz}g(z) = slope\ of\ g(z)\ at\ z$$

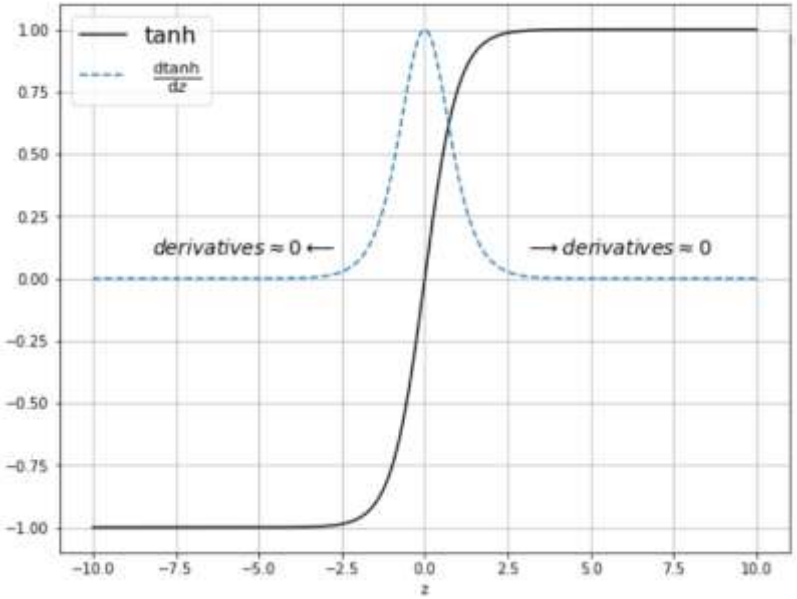
$$\frac{d}{dz}g(z) = \frac{(e^z+e^{-z})(e^z+e^{-z})-(e^z-e^{-z})(e^z-e^{-z})}{(e^z+e^{-z})^2} = \frac{(e^z+e^{-z})^2-(e^z-e^{-z})^2}{(e^z+e^{-z})^2}$$

$$\frac{d}{dz}g(z) = \frac{\frac{(e^z+e^{-z})^2-(e^z-e^{-z})^2}{(e^z+e^{-z})^2}}{\frac{(e^z+e^{-z})^2}{(e^z+e^{-z})^2}} = \frac{\frac{1-tanh(z)^2}{1}}{1} = 1 - tanh(z)^2$$

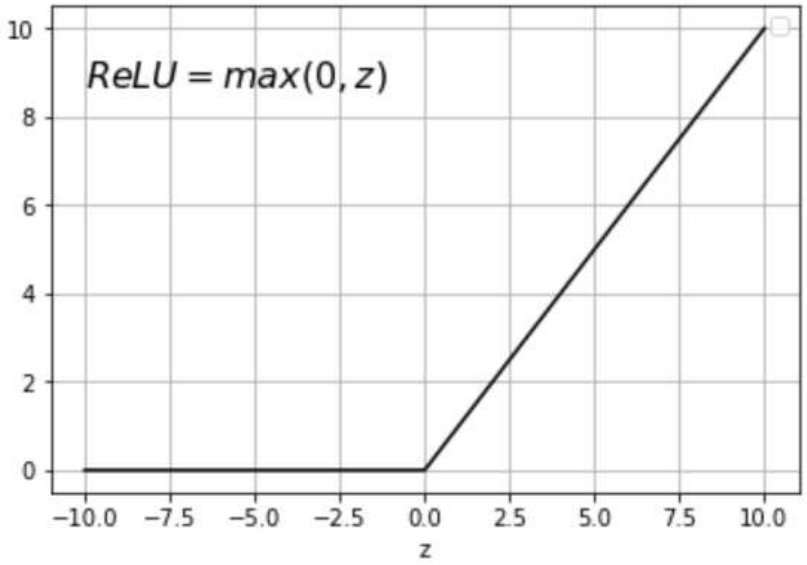
$$z = 10 \quad tanh(z) \approx 1 \Rightarrow \frac{d}{dz}g(z) \approx 1-1^2 \approx 0$$

$$z = -10 \quad tanh(z) \approx -1 \Rightarrow \frac{d}{dz}g(z) \approx 1-(-1)^2 \approx 0$$

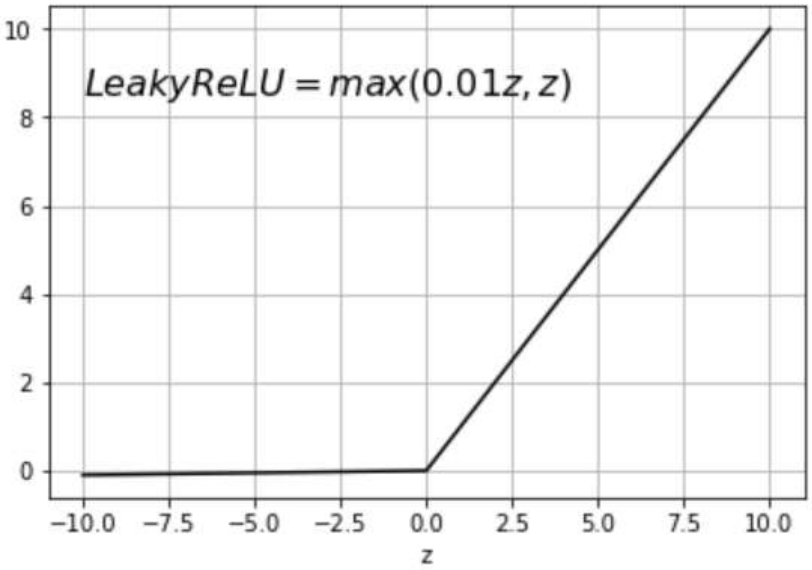
$$z = 0 \quad tanh(z) = 0 \Rightarrow \frac{d}{dz}g(z)(z) = 1 - 0^2 = 1$$



ReLU and LeakyReLU activation function



ReLU function

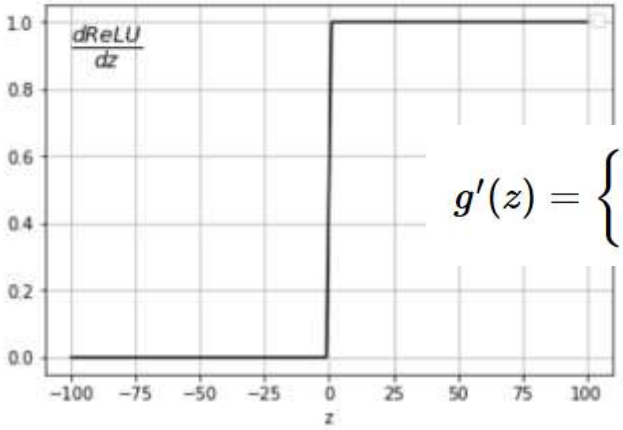


LeakyReLU function

Derivatives of ReLU and LeakyReLU activation functions

$$g(z) = max(0, z)$$

$$g'(z) = \begin{cases} 1 \ if \ z > 0 \\ 0 \ if \ z < 0 \\ undefined \ if \ z = 0 \end{cases}$$

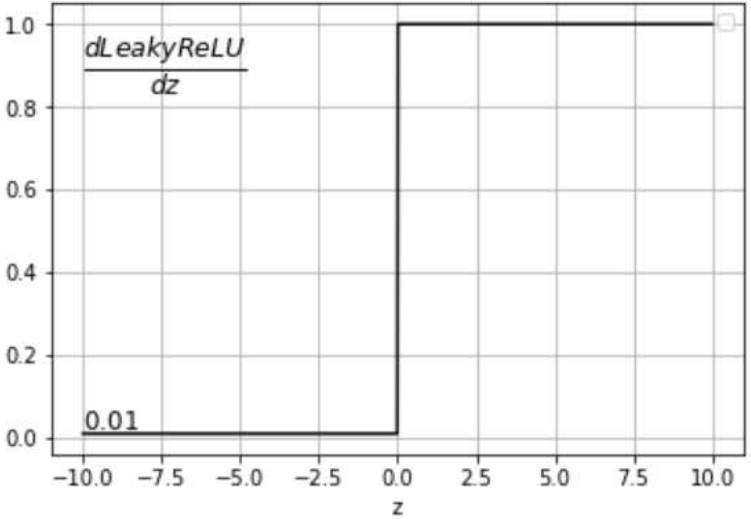


$$g'(z) = \begin{cases} 0 & if \ z < 0 \\ 1 & if \ z \geq 0 \end{cases}$$

Derivative of a $ReLU$ function

$$g(z) = max(0.01z, z)$$

$$g'(z) = \begin{cases} 0.01 & if \ z < 0 \\ 1 & if \ z \geq 0 \end{cases}$$



Derivative of a $LeakyReLU$ function

Why the non-linear activation function?

- If we want our activation functions to be linear functions, so that we have g[1]=z[1] and g[1]=z[1] , then these equations above become:
- This loses much of the representational power of the neural network as often times the output that we are trying to predict has a non-linear relationship with the inputs.

$$z^{[1]} = W^{[1]}x + b^{[1]}$$

$$a^{[1]} = g^{[1]}(z^{[1]})$$

$$z^{[2]} = W^{[2]}x + b^{[2]}$$

$$a^{[2]} = g^{[2]}(z^{[2]})$$

$$z^{[1]} = W^{[1]}x + b^{[1]}$$

$$a^{[1]} = g^{[1]}(z^{[1]})$$

$$a^{[1]} = z^{[1]} = W^{[1]}x + b^{[1]}$$

$$a^{[2]} = z^{[2]} = W^{[2]}a^{[1]} + b^{[2]}$$

$$a^{[2]} = W^{[2]}(W^{[1]}x + b^{[1]}) + b^{[2]}$$

$$a^{[2]} = (W^{[2]}W^{[1]})x + (W^{[2]}b^{[1]} + b^{[2]}) = W'x + b'$$

Why the non-linear activation function?

- It's clear that if we use a linear activation function (identity activation function), then the Neural Network will output linear output of the input. This loses much of the representational power of the neural network as often times the output that we are trying to predict has a non-linear relationship with the inputs.
- It can be shown that if we use a linear activation function for a hidden layer and sigmoid function for an output layer, our model becomes a logistic regression model. Due to the fact that a composition of two linear functions is a linear function, our area of implementing such a Neural Network reduces rapidly. Rare implementation example can be solving a regression problem in machine learning (where we use a linear activation function in a hidden layer). Recommended usage of linear activation function is to be implemented in the output layer in case of regression.

Warm up: Gradient Descent

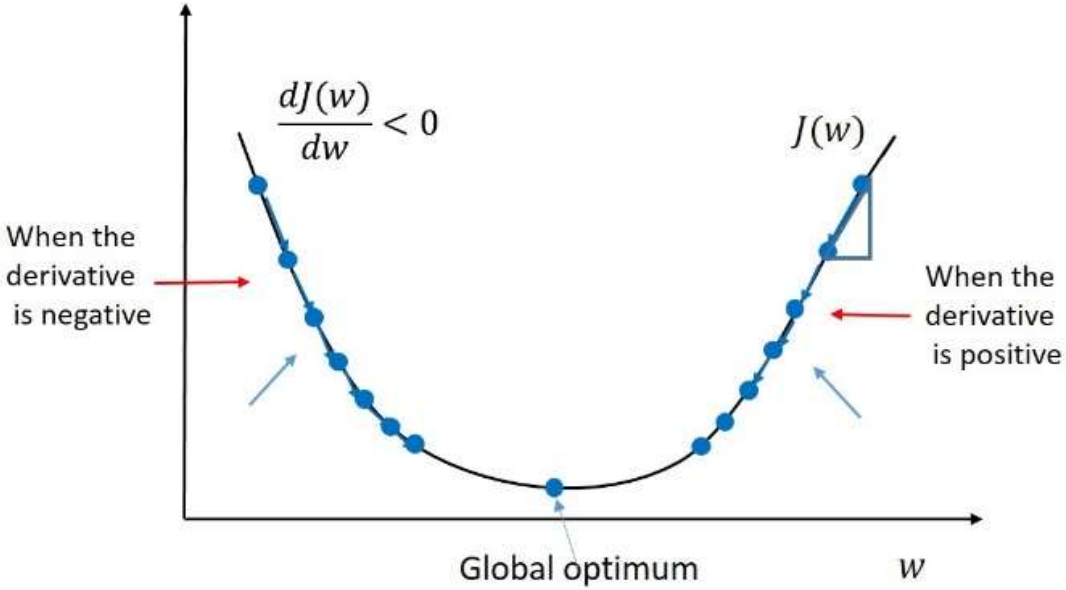
- Gradient Descent: A term “Deep learning” refers to training neural networks and sometimes very large neural networks. They are massively used for problems of classification and regression. The main goal is to optimize parameters w and b (both in logistic regression and in neural networks).

- developed in the 1940s, 1950s and 1960s.

- Gradient Descent is an algorithm that tries to minimize the error (cost) function J(w,b) and to find optimal values for w and b.

FSB

Warm up: Gradient Descent



$$w = w - \alpha \frac{\mathrm{d}J(w,b)}{\mathrm{d}w}$$

$$b = b - \alpha \frac{\mathrm{d}J(w,b)}{\mathrm{d}b}$$

Gradient Descent for Neural Networks

FSB

Gradient Descent for Neural Networks

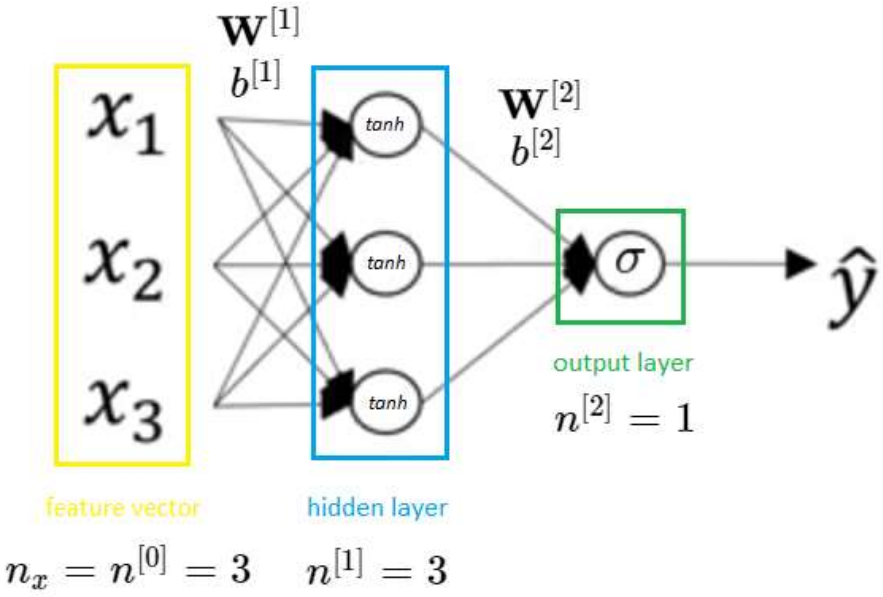
- How to implement gradient descent for one hidden layer Neural Network?

- $\mathbf{W}^{[1]}$ is $(n^{[1]}, n^{[0]})$ matrix
- $b^{[1]}$ is $(n^{[1]}, 1)$ matrix or a column vector
- $\mathbf{W}^{[2]}$ is $(n^{[2]}, n^{[1]})$ matrix
- $b^{[2]}$ is $(n^{[2]}, 1)$, so far $b^{[2]}$ is a scalar

Equations for one example x(i) :

$$z^{[1](i)} = \mathbf{W}^{[1]} x^{(i)} + b^{[1]}$$
$$a^{[1](i)} = \tanh(z^{[1](i)})$$
$$z^{[2](i)} = \mathbf{W}^{[2]} a^{[1](i)} + b^{[2]}$$
$$\hat{y}^{(i)} = a^{[2](i)} = \sigma(z^{[2](i)})$$

$$\hat{y}^{(i)} = \begin{cases} 1 & \text{if } a^{[2](i)} > 0.5 \\ 0 & \text{otherwise} \end{cases}$$



One hidden layer Neural Network

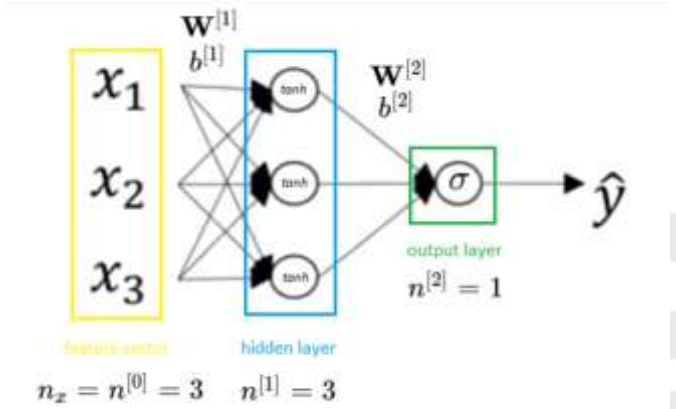
Gradient Descent for Neural Networks

- How to implement gradient descent for one hidden layer Neural Network?

Equations for one example x(i) :

$$z^{[1](i)} = \mathbf{W}^{[1]}x^{(i)} + b^{[1]}$$
$$a^{[1](i)} = \tanh(z^{[1](i)})$$
$$z^{[2](i)} = \mathbf{W}^{[2]}a^{[1](i)} + b^{[2]}$$
$$\hat{y}^{(i)} = a^{[2](i)} = \sigma(z^{[2](i)})$$

$$\hat{y^{(i)}} = \begin{cases} 1 & \text{if } a^{[2](i)} > 0.5 \\ 0 & \text{otherwise} \end{cases}$$



One hidden layer Neural Network

$$J(\mathbf{W}^{[1]}, b^{[1]}, \mathbf{W}^{[2]}, b^{[2]}) = \frac{1}{m}\sum_{i=1}^{m}\mathcal{L}(\hat{y}^{(i)}, y^{(i)}) = \frac{1}{m}\sum_{i=1}^{m}\mathcal{L}(a^{[2](i)}, y^{(i)})$$

$$J = -\frac{1}{m}\sum_{i=0}^{m}\left(y^{(i)}\log\left(a^{[2](i)}\right) + (1-y^{(i)})\log\left(1-a^{[2](i)}\right)\right)$$

Gradient Descent for Neural Networks

- To train parameters of our algorithm we need to perform gradient descent. When training neural network, it is important to initialize the parameters randomly rather then to all zeros.
- So after initializing the parameters we get into gradient descent which looks like this:

$$
\begin{aligned}
&\textit{repeat until convergence}: \\
&\qquad \textit{calculate predictions } (\hat{y}, i = 1..m) \\
&\qquad \mathbf{dW}^{[1]} = \frac{\partial J}{\partial \mathbf{W}^{[1]}} \\
&\qquad db^{[1]} = \frac{\partial J}{\partial db^{[1]}} \\
&\qquad \mathbf{dW}^{[2]} = \frac{\partial J}{\partial \mathbf{W}^{[2]}} \\
&\qquad db^{[2]} = \frac{\partial J}{\partial db^{[2]}} \\
&\qquad \mathbf{W}^{[1]} = \mathbf{W}^{[1]} - \alpha \mathbf{dW}^{[1]} \\
&\qquad b^{[1]} = b^{[1]} - \alpha db^{[1]} \\
&\qquad \mathbf{W}^{[2]} = \mathbf{W}^{[2]} - \alpha \mathbf{dW}^{[2]} \\
&\qquad b^{[2]} = b^{[2]} - \alpha db^{[2]}
\end{aligned}
$$

Gradient Descent for Neural Networks

- Forward propagation equations (remember that if we are doing a binary classification then the activation function in the output layer is a sigmoid function):

Non – Vectorized (for a single training example)	Vectorized
$z^{[1](i)} = \mathbf{W}^{[1]}x^{(i)} + b^{[1]}$	$\mathbf{Z}^{[1]} = \mathbf{W}^{[1]}X + b^{[1]}$
$a^{[1](i)} = \tanh(z^{[1](i)})$	$\mathbf{A}^{[1]} = \tanh(\mathbf{Z}^{[1]})$
$z^{[2](i)} = \mathbf{W}^{[2]}a^{[1](i)} + b^{[2]}$	$\mathbf{Z}^{[2]} = \mathbf{W}^{[2]}A^{[1]} + b^{[2]}$
$\hat{y}^{(i)} = a^{[2](i)} = \sigma(z^{[2](i)})$	$\mathbf{A}^{[2]} = \sigma(\mathbf{Z}^{[2]}) = \sigma(\mathbf{Z}^{[2]})$

Gradient Descent for Neural Ne

$$\mathbf{Z}^{[1]} = \mathbf{W}^{[1]}X + b^{[1]}$$

$$\mathbf{A}^{[1]} = \tanh(\mathbf{Z}^{[1]})$$

$$\mathbf{Z}^{[2]} = \mathbf{W}^{[2]}A^{[1]} + b^{[2]}$$

$$\mathbf{A}^{[2]} = \sigma(\mathbf{Z}^{[2]}) = \sigma(\mathbf{Z}^{[2]})$$

- We will show equations in the backpropagation step:

Sign $*$ stands for element wise multiplication

Non – Vectorized

$$dz^{[2]} = a^{[2]} - y$$

$$\mathbf{dW}^{[2]} = dz^{[2]}a^{[1]T}$$

$$db^{[2]} = dz^{[2]}$$

$$dz^{[1]} = \mathbf{W}^{[1]T}dz^{[2]} * g'^{[1]}(z^{[1]})$$

$$\mathbf{dW}^{[1]} = dz^{[1]}x^T$$

$$db^{[1]} = dz^{[1]}$$

$$\mathbf{dZ}^{[2]} = \mathbf{A}^{[2]} - \mathbf{Y}$$

$$\mathbf{dW}^{[2]} = \mathbf{dZ}^{[2]}\mathbf{A}^{[1]T}$$

$$db^{[2]} = \frac{1}{m}np.sum(\mathbf{dZ}^{[2]}, axis = 1, keepdims = True)$$

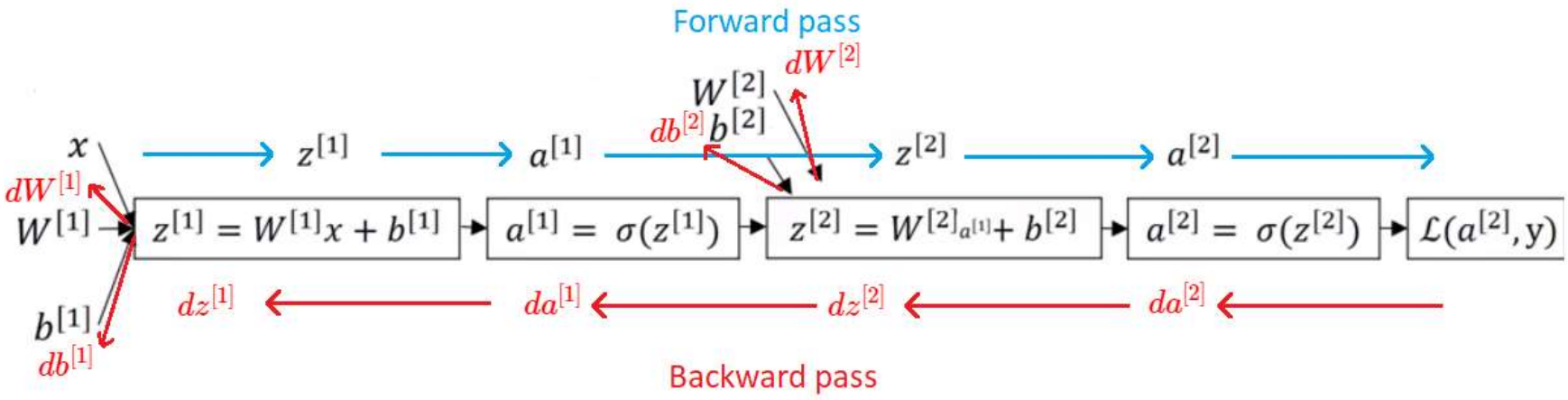
$$\mathbf{dZ}^{[1]} = \mathbf{W}^{[1]T}\mathbf{dZ}^{[2]} * g'^{[1]}(\mathbf{Z}^{[1]})$$

$$\mathbf{dW}^{[1]} = \frac{1}{m}\mathbf{dZ}^{[1]}\mathbf{X}^T$$

$$db^{[1]} = \frac{1}{m}np.sum(\mathbf{dZ}^{[1]}, axis = 1, keepdims = True)$$

Gradient Descent for Neural Networks

- **Backpropagation graph for one hidden layer Neural Network**



Computation graph for 2 – layer Neural Network

Example Implementations

- Logistic Regression training from scratch

https://colab.research.google.com/drive/1XoXwet_hvNQK7LIv_NHbONwVim9AmvcM?usp=sharing

- Logistic Regression training from SKLearn

https://colab.research.google.com/drive/1MnB7kcgu2YPfVU61oLQy2z0O-jxA6xwM?usp=sharing

- Logistic Regression training fast using vectorization

https://colab.research.google.com/drive/1LZyvv0gVLt1kTuBx1YzHbYCh7xudzMUe?usp=sharing

FSB